LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

Introduction

Requirements

Costruire un sistema software distribuito costituio da N (N>=1) Produttori che inviano informazione a 1 Consumatore, il quale deve elaborare tale informazione. La dislocazione dei componenti sui nodi di elaborazione può essere: OneNode: tutti i componenti operano nello stesso nodo; TwoNodes: gli N Produttori operano in uno stesso nodo, mentre il Consumatore opera in un diverso nodo; ManyNodes: il Consumatore opera in suo proprio nodo, mentre i Produttori operano su K nodi diversi (1 < K <=N).

Requirement analysis

- R1: I produttori inviano informazione al consumatore che la elabora
- R2: n. produttori >= 1
- R3: n. consumatori = 1
- R4: modalità OneNode := produttori e consumatore sullo stesso nodo
- R5: modalità TwoNodes := 1 nodo per consumatore + 1 nodo per tutti i produttori
- R6: modalità ManyNodes := 1 nodo per consumatore + K nodi per i produttori (1 < K <=N)

Problem analysis

Da R1,R2 ed R3 intuiamo che il sistema è composto da due tipologie di entità ovvero Produttore e Consumatore, ci sarà almeno un Produttore ed esattamente un Consumatore.
I Produttori devono essere in grado di comunicare con il Consumatore, indipendentemente che questi si trovi sul loro stesso nodo (R4) o su un altro nodo (R5,R6).
Da R2 ed R3 si evince che essendoci 1 Consumatore che comunica con N Produttori risulta applicabile il modello Cliente/Servitore, con i Produttori come clienti ed il consumatore come Servitore. I requisiti non prevedono l'invio di risposte da parte del Consumatore si puo' dunque anche pensare di utilizzare la variante del Cliente/Servitore con clienti non bloccanti.

Test plans

- Elaborazione: far elaborare al consumatore una serie di valori fake dei quali si conosce l'output corretto per verificare che l'algoritmo di elaborazione sia corretto
- Comunicazione: far stampare al Consumatore i valori Ricevuti da N Produttori nei seguenti casi
  - OneNode: Consumatore e Produttori sono su uno stesso nodo virtuale

- TwoNodes: il Consumatore è da solo su nodo virtuale mentre gli N Produttori sono tutti su un'altro nodo virtuale distinto
- ManyNodes: il Consumatore è da solo su un nodo virtuale mentre N Produttori sono divisi tra K nodi virtuali

in modo da testare tutte le possibili modalità di comunicazione

Project

Come primo sprint si vuole implementare una versione di base del sistema.
Si sceglie in prima istanza di adottare un' architettura client-server in cui Produttore e Consumatore sono due eseguibili java.
Per la comunicazione tra i due componenti si sceglie di utilizzare l'astrazione di Interaction fornita dalla libreria unibo.basicomm23-1.0.
Nello specifico si vuole adottare un'Interaction che abbia le seguenti proprietà:

- sia basata su protocollo tcp in modo da garantire un ragionevole livello di sicurezza per quanto riguarda la ricezione dei messaggi
- sia presente una semantica sincrona-bloccante tra Produttore e Consumatore, nello specifico il protocollo di comunicazione tra Produttore e Consumatore prevederà la sequenza di messaggi:
  1. Produttore -> Consumatore : msg(sendInfo,request,prod,cons,info,seq)
  2. Consumatore: elaborazione messaggio
  3. Consumatore -> Produttore : msg(ack,reply,cons,prod,ack,seq)

Per il consumatore si sceglie di fare uso di un componente IServer che lo abiliterà alla ricezione dei messaggi.
La logica per l'elaborazione delle richeste e l'invio delle risposte viene inserita all'interno di un componente ConsApplMsgHandler che implementa l'interfaccia IApplMsgHandler.
Non avendo informazioni precise sul tipo di elaborazione da eseguire nel prototipo il Consumatore si limiterà a stampare a video la richiesta ricevuta prima di inviare l'ack.
n.b.: la scelta di adottare un comunicazione sincrona è dettata dalla volontà di garantire al Produttore un feedback sia sulla ricezione che sulla corretta elaborazione dei messaggi.
In questo primo sprint si è scelto inoltre di adottare una semantca bloccante in modo da poter produrre un primo prototipo in tempi ragionevoli, tuttavia se i primi test sul sistema saranno soddisfacenti si puo' valutare di introdurre nei prossimi sprint un proxy tra Produttore e Consumatore in modo da rendere la comunicazione sincrona-non bloccante.

Testing

Deployment

Maintenance

By Student: Alessio Benenati
Email: alessio.benenati2@studio.unibo.it

GIT repo: https://github.com/alessioBenenatiUnibo/IngegneriaSistemiSW